

sqlsh

A Drop-In, Completion-Aware SQLite Shell — Design and Implementation

sqlsh project

2026-09-25

Table of contents

1	Introduction	3
1.1	Scope and non-goals	3
2	Why a clean-room reimplementation	4
2.1	Dependencies	4
3	Module decomposition	5
3.1	Connection settings and correctness	6
3.2	No globals below <code>main.c</code>	6
4	The completion pipeline	7
4.1	Stage 1: context analysis	7
4.2	Stage 2: candidate generation	7
4.3	Stage 3: menu presentation	8
4.4	Value completion	8
5	Highlighting and theming	9
5.1	The theme system	9
6	Terminal handling	11
7	Error handling conventions	12
8	Output modes and parity	13
9	Development methodology	14
9.1	The gate	14
9.2	The manual-check gate	15
9.3	Version control conventions	15
10	Manual verification	16
11	Using <code>sqlsh</code>	17
11.1	Building	17
11.2	Basic usage	17
11.3	Keybindings	17
11.4	Reference documentation	18
12	Summary	20

1 Introduction

`sqlsh` is a from-scratch, C99 reimplementation of the `sqlite3(1)` command-line shell. It is built to be a **drop-in replacement**: every dot command, output mode, and command-line flag that upstream `sqlite3(1)` accepts is either implemented to the same behaviour or refused with an explicit message — never silently ignored. `sqlsh --compat` reproduces upstream exactly, verified byte-for-byte by a differential test suite that runs the same input through both binaries and compares the output.

Outside `--compat`, `sqlsh` adds what a modern interactive shell is expected to have and upstream's `shell.c` does not: a navigable, `zsh`-style completion menu; live syntax highlighting of the statement as it is typed; a colour theme system; and box-drawn, terminal-width-aware output by default. The design premise is that `alias sqlite3=sqlsh` should survive a normal day's work — nothing about scripting or automation should notice the substitution, while interactive use gets meaningfully better.

This document explains what the program does, how it is built internally, and why its structure looks the way it does. It draws together the project's architecture notes, its development methodology, its manual testing guide, and its reference manual into one continuous account, for a reader meeting the codebase for the first time.

1.1 Scope and non-goals

Stating what `sqlsh` deliberately does not attempt keeps the rest of the design legible:

- **No bundled SQLite extensions.** Upstream `shell.c` carries `sqlar`, `zipfile`, `sha3`, `sqlite3expert`, `sqlite3recover`, `sqlite3_intck`, `sqlite3session`, and `dbdata` — together the bulk of its 37,000-plus lines. The dot commands that depend on them are recognised and refused with a message naming the missing extension, so the gap is documented rather than silently discovered at the worst moment.
- **No SQL parser.** Completion needs to know *where the cursor is* inside a statement, not what the statement means. A tokenizer and a cursor-context state machine are sufficient; a full parser would be scope the feature does not need.
- **No Windows support.** POSIX `termios` is assumed throughout the terminal layer.

2 Why a clean-room reimplementation

The upstream client, `shell.c`, is 37,373 lines and over a megabyte in the pinned SQLite 3.53.3 — machine-generated by concatenating `src/shell.c.in` with a dozen extension sources. It cannot be meaningfully diffed, re-derived, or hand-maintained, and grafting a completion engine onto it would mean owning a blob nobody in this project wrote.

`sqlsh` is therefore built against the public `sqlite3` C API rather than as a textual fork. Upstream `shell.c` is kept for consultation only: `make reference` copies it into `reference/`, sourced from the same amalgamation the Nix flake already pins — so the project names one SQLite version, not two — and the copy is excluded from git. When an upstream behaviour must be matched exactly (a quoting rule, a dot command's flag set, the precise column-width algorithm), it is read there and reimplemented deliberately.

The tradeoff this accepts is that parity must be *built and verified* deliberately rather than inherited for free. It is measured, not claimed: the gate runs a differential suite that executes identical input through `sqlsh --compat` and the `sqlite3` binary from the flake and diffs the two outputs byte for byte. What is gained in return is that every line under `src/` is the project's own, small, and auditable.

2.1 Dependencies

`sqlsh` depends on exactly one library beyond `libc` and POSIX: `libsqlite3`.

Notably absent is any line-editing library. This is a deliberate consequence of the feature set, not asceticism: GNU `readline`, `libedit`, and `linenoise` all implement completion as *print a list of candidates and return to the prompt*. None has a concept of a persistent, navigable selection menu that narrows as the user keeps typing. Reproducing `zsh`'s completion behaviour means owning the terminal outright, so `sqlsh` does, in `line.c` and `menu.c`.

Two build paths exist for SQLite itself, and both are exercised by the gate:

- `make` links the system `libsqlite3`, located by `pkg-config`. Because the Nix flake pins `nixpkgs`, this path is reproducible across machines.
- `make SQLITE=vendored` compiles the SQLite amalgamation directly, producing a binary with no external SQLite dependency at all. The amalgamation itself is fetched by the flake and never committed — 9.4 MB and roughly 266,000 lines — and the flake's pinned amalgamation version is kept equal to `pkgs.sqlite`'s, with the dev shell warning if they drift apart.

Build objects land in `build/<backend>/<mode>/`, and the directory is keyed on both dimensions deliberately: a single shared `build/` tree lets `make test` link objects an earlier `make debug` compiled without sanitizers, which would let the suite pass while running uninstrumented — keying by mode closes that hole.

3 Module decomposition

Each module is one `.c` file in `src/` with a matching header in `include/`. Each owns exactly one concern and depends only on the modules below it in the stack; no module reaches into another's internals.

```
main.c      argument parsing (upstream flag set) and nothing else
|
+-- shell.c  the session: the current connection and the auxiliary ones,
|            |            the .output/.once redirect, the switches, statement
|            |            execution, the REPL, the init files
|            |
|            +-- dot.c      the command table and dispatch: 76 rows,
|            |            64 implemented and 12 refused
|            +-- schema.c  .schema .fullschema .dump .databases .indexes .tables
|            |            .dbinfo .dbtotxt .clone .lint -- byte-exact ports
|            +-- import.c  .import (RFC 4180 CSV and ASCII-delimited), .excel/.www
|            |
|            +-- out.c      the 22 output-mode presets, box drawing, type-aware colour
|            |
|            +-- width.c    UTF-8 display width, also used by menu.c
|
+-- line.c    raw-mode line editor: keys, cursor, history, redraw
|            |
|            +-- hl.c       syntax highlighting of the buffer being typed
|            +-- menu.c     completion menu: layout, navigation, rendering
|            |
|            +-- comp.c     candidate generation: asks sqlctx where we are,
|            |            |            asks db what exists, returns a candidate list
|            +-- sqlctx.c   SQL tokenizer + cursor-context state machine
|
+-- theme.c   colour table, terminal capability detection, theme file
+-- db.c      sqlite3 handle, statement execution, schema introspection,
              the value cache
```

Two placements in this diagram encode load-bearing decisions rather than convenience.

First, `out.c` sits **under** `db.c`, not above it. `db.c` hands the formatter typed values and column names and decides nothing about appearance, and nothing above `db.c` includes `<sqlite3.h>` at all. That separation is what lets the formatter be unit-tested against synthetic rows with no database attached, and it is also what lets the differential parity suite treat the entire pipeline, from query text to output bytes, as a single pure function.

Second, `sqlctx.c` and `comp.c` are pure with respect to the terminal: each takes a buffer and a cursor offset and returns data, touching no global state and no I/O. That purity is what makes the completion engine unit-testable without a pseudo-terminal, and it is the main reason the module boundaries were drawn where they were.

3.1 Connection settings and correctness

`db.c` applies upstream's connection settings itself, both on first open and on every `.open`: double-quoted string literals off for DDL and DML, defensive mode on, trusted-schema off, statement scan-status off, extension loading enabled. These are behavioural choices, not decoration — with double-quoted-string-literals off, `SELECT "typo"` is a compile error rather than a string literal that silently returns the wrong thing.

Disabling that setting explicitly matters because upstream `sqlite3(1)` compiles its own SQLite with `-DSQLITE_DQS=0`, while `sqlsh` links whichever shared `libsqlite3` the system provides, which may or may not have been built the same way. Setting it in `db.c` makes the two shells behave identically regardless of which library sits underneath either one.

3.2 No globals below `main.c`

A dot command is handed a `Shell` struct and reaches everything else through it — there is no global mutable state anywhere below `main.c`. The redirect a command writes to, for instance, is always `shell_out(sh)`: the `.output` file while one is active, the base stream otherwise. This is also why `.once` needs no special-casing in the REPL loop — the shell simply ends the redirect after the next statement runs.

4 The completion pipeline

A <Tab> keystroke at cursor offset n in buffer b flows through three stages.

4.1 Stage 1: context analysis

`sqlctx_analyze(b, n)` tokenizes the buffer — respecting string literals, `--` and `/* */` comments, and quoted identifiers (`"x"`, ``x``, `[x]`) — then walks the resulting token stream to produce a context structure:

```
typedef struct {
    SqlCtxKind kind;          /* what may appear here */
    const char *prefix;       /* the partial word under the cursor */
    size_t prefix_len;
    TableRef tables[SQLCTX_MAX_TABLES]; /* from FROM/JOIN, with aliases */
    size_t table_count;
    char column[SQLCTX_MAX_IDENT]; /* for VALUE contexts */
} SqlContext;
```

`kind` takes one of nine values: `CTX_DOT_COMMAND`, `CTX_DOT_ARG`, `CTX_STATEMENT_START`, `CTX_SELECT_LIST`, `CTX_TABLE`, `CTX_COLUMN`, `CTX_VALUE`, `CTX_KEYWORD`, or `CTX_UNKNOWN`.

4.2 Stage 2: candidate generation

`comp_candidates(ctx, db)` maps that context to a source of candidates:

Context	Candidates
CTX_DOT_COMMAND	the dot-command table in <code>dot.c</code>
CTX_DOT_ARG	per-command: table names, modes, filenames
CTX_STATEMENT_START	statement keywords (<code>SELECT</code> , <code>INSERT</code> , ...)
CTX_SELECT_LIST	columns of tables already named in <code>FROM</code> ; else all
CTX_TABLE	tables and views
CTX_COLUMN	columns scoped to the <code>FROM</code> set, alias-qualified
CTX_VALUE	distinct values of the column being compared
CTX_KEYWORD	keywords valid after the preceding token

Scoping is the entire point of this stage: `SELECT * FROM companies WHERE <Tab>` must offer the seven columns that belong to `companies`, not every column in the database. Table-scoped completion is what separates this from a naive identifier-matching completer.

4.3 Stage 3: menu presentation

`menu_show(candidates)` renders the candidate list below the prompt and takes over input handling. `Tab`, `→`, and `Ctrl-N` move to the next candidate; `Shift-Tab`, `←`, and `Ctrl-P` move to the previous one; `Enter` accepts the selection; `Esc` or `Ctrl-G` dismisses the menu. Typing while the menu is open narrows the candidate list in place rather than closing it — matching the `zsh` behaviour the feature is modeled on. A context with exactly one candidate is inserted directly, with no menu drawn at all.

4.4 Value completion

`CTX_VALUE` is the only context that reads user data rather than schema metadata, so it is deliberately bounded on every axis:

- The underlying query is `SELECT DISTINCT <col> FROM <tbl> LIMIT 200`, with identifiers quoted.
- A `sqlite3_progress_handler` callback aborts the query after roughly 150 ms, producing no menu rather than a stalled prompt.
- Results are cached per `(table, column)` pair for the session, and the cache is invalidated whenever a statement that modifies the database runs.
- When the 200-row cap is hit, the menu says so explicitly, so a truncated list is never mistaken for a complete one.

Value completion is on by default; `.complete values off` disables it for sessions against databases where even a capped scan is unwelcome.

5 Highlighting and theming

`hl_write(out, text, from, to, schema)` writes a window of the line currently being typed, coloured token by token. It reuses `sqlctx.c`'s lexer rather than maintaining a second one — two independent lexers would eventually drift, and highlighting would stop agreeing with completion about what a token is. `hl_write` always analyses the *whole* line, even when only part of it is being redrawn, because a string literal that opened before the visible window still needs to colour everything after it correctly.

`hl.c` itself knows nothing about a database; schema knowledge arrives through four callbacks:

```
typedef struct {
    bool (*is_keyword)(void *ctx, const char *name, size_t len);
    bool (*is_function)(void *ctx, const char *name, size_t len);
    bool (*is_table)(void *ctx, const char *name, size_t len, bool *is_view);
    bool (*is_column)(void *ctx, const char *name, size_t len,
                      const SqlCtxTable *tables, size_t ntables);
    void *ctx;
} HlSchema;
```

`main.c` supplies the concrete callbacks; `line.c` merely carries the struct through — the same injection pattern used for the completer, and for the same reason: nothing above `db.c` may include `<sqlite3.h>`, and every colouring decision becomes independently testable against a stub implementation. Each callback is asked once per identifier per keystroke and never queries the database directly — it answers from a cache `db.c` already maintains and invalidates on the schema cookie. Columns are looked up only within the tables that `sqlctx_analyze` reports for the current statement, which is why `SELECT id` leaves `id` in the plain foreground while `SELECT id FROM users` colours it as a known column.

The payoff of this design is direct: a name the database does not recognise stays in the terminal's plain foreground colour, making a typo visible before the statement is ever executed.

5.1 The theme system

Nothing in the program hardcodes a colour. `theme.c` holds exactly one SGR (ANSI Select Graphic Rendition) escape string per style, and the four built-in palettes — `default`, `dark`, `light`, and the 16-colour-safe `basic` — exist as theme-file `text` compiled into the binary, parsed by the same code path a user-supplied theme file goes through. There is therefore nothing a shipped theme can do that a user's theme file cannot also do, and nothing to install separately.

A theme file is an INI-style document: `[menu]`, `[syntax]`, and `[output]` sections containing `key = value` lines, where a value is zero or more text attributes plus at most one foreground and one background colour — for example `bold blue`, `dim`, `underline #ff0080 on 52`, or `none`. A line whose first non-blank character is `#` is a comment, and `--` comments to end of line as in SQL; `#` cannot serve double duty as a SQL-style comment marker, because a value is the one place a `#rrggbb` colour literal legitimately appears. Unknown sections, keys, or values are

reported and skipped individually — a typo in a theme file costs the user that one line, never the whole palette and never the shell’s ability to start.

`theme_dump` writes the live, in-memory palette back out in exactly this format, so `.theme` with no argument is simultaneously a listing and a ready-to-edit starting point; a test asserts the round trip for every shipped palette. `.theme` also accepts `list`, `reload`, `on/off`, a built-in name, or a file path. The theme file itself lives at `$XDG_CONFIG_HOME/sqlsh/theme` (defaulting to `~/.config/sqlsh/theme`), and `-noinit` suppresses it exactly as it suppresses `~/.sqliterc`.

6 Terminal handling

`line.c` places the terminal into raw mode with `tcsetattr`, and restores it both from an `atexit` handler and from `SIGINT`, `SIGTERM`, `SIGQUIT`, and `SIGHUP` handlers — a shell that leaves a user’s terminal stuck in raw mode after a crash is a worse outcome than crashing with a clean terminal. `SIGWINCH` triggers a re-layout of any open completion menu, so a resized window does not leave stale rows on screen.

Output uses plain ANSI escape sequences directly; there is no dependency on `terminfo` or `ncurses`. The escape set in use is small and universally supported: cursor movement, erase-line, erase-to-end-of-screen, and SGR colour codes. When standard output is not a terminal, `sqlsh` reads lines with `fgets` and performs no editing at all, so piping input through the shell continues to work exactly as it would with any other batch tool.

Two edge cases are tracked deliberately rather than papered over: input lines longer than the terminal’s width, and multi-line statement continuation. UTF-8 display width — needed for both of these, and for aligning box-drawn output around wide or combining characters — has its own module, `width.c`, shared between the output formatter and the completion menu.

7 Error handling conventions

The codebase follows a small, consistent set of rules for failure paths:

- Functions that can fail return `bool` or a negative `int` — never a sentinel pointer value the caller is expected to compare against something magic.
- Allocation failure is reported upward to the caller, never treated as fatal, except inside `main` itself.
- Every `sqlite3_stmt` is finalized on every code path, including error paths — there is no path that leaks a prepared statement.
- No global mutable state exists outside `main.c`'s single `Shell` struct, which every function that needs it receives explicitly as a parameter.

8 Output modes and parity

The default output mode, whether the session is interactive or not, is **box**: headers on, colour on, sized to the real terminal width via `ioctl(TIOCGWINSZ)`, re-read before every result so that resizing the window takes effect starting with the next query. `--compat` switches the default to **list**, headers off, no colour — upstream’s own default.

Thirteen “classic” output modes (`ascii` `box` `column` `csv` `html` `insert` `json` `line` `list` `markdown` `quote` `table` `tabs`) are byte-parity-tested against `sqlite3(1)` by `make parity`; nine further modes (`c` `count` `jatom` `jobject` `off` `psql` `qbox` `split` `tcl`) are best-effort, since they either have no clean upstream equivalent to diff against or are rarely used enough that hand verification suffices. Inside the shell, `.help mode` lists the option flags each mode accepts.

Where `sqlsh` knowingly diverges from upstream — an extension this build does not vendor, or an output mode the shipped `sqlite3(1)` binary itself rejects — the gap is refused loudly, with an error naming exactly what is missing, rather than approximated silently. `docs/notes/` and the project’s internal roadmap carry the complete, current list of such gaps; inside the running shell, `.help` marks each of the eleven currently-refused dot commands as such.

9 Development methodology

The project follows a specification-driven workflow designed for an autonomous coding agent working under human supervision, structured around a small number of principles.

Ground truth lives on disk, never only in conversation. A project tracking file is authoritative, and its roadmap checkboxes are the state of record; a fresh working context — after a restart, a new session, or handing work to a sub-agent — resumes from nothing but “execute phase N ” plus that file.

Planning is supervised; implementation is autonomous. Design decisions are resolved with the project owner at planning time and recorded in the roadmap. Execution then proceeds one phase per fresh context, without a human in the loop, except at the manual-check gate described below.

An undefined decision is flagged, never guessed. If a phase’s plan is silent on a question that implementation needs answered, that is a stop condition, not an invitation to improvise.

Nothing is tracked outside the roadmap. Work discovered mid-phase — a bug, a missing edge case, a deferred improvement — is recorded in the roadmap itself, as a task, a deferred-work entry, or a new phase; it is never carried only in conversation or left implicit.

9.1 The gate

Every phase is certified by one command producing one of three verdicts: PASS, FAIL, or BLOCKED. The gate, not any agent’s self-assessment, is the sole arbiter of whether a phase is done.

A green result requires, simultaneously:

- a clean formatter run,
- a clean linter run,
- a clean type/compiler check,
- **zero warnings** across the build and every check above — a warning is treated as a defect, fixed at its cause or explicitly waived in the roadmap with a written rationale, never suppressed silently or globally, and
- at least one acceptance test that actually **passed**, not merely collected or skipped — a skipped test does not count as a pass.

Concretely, in this repository, `make gate` runs the formatter, both the debug and release builds under `-Werror`, the ASan/UBSan-instrumented test suite, the differential parity suite, `cppcheck`, and `clang-tidy`, in that order, stopping at the first failure.

9.2 The manual-check gate

Some behaviour cannot be honestly verified by an automatic check: visual menu placement, whether colours read legibly on a real terminal, whether a resize behaves correctly on real hardware. Any phase that carries such a manual check is completed up through a green automatic gate, has its roadmap entry marked in-progress, and then **halts** — the agent does not commit. The project owner performs the listed manual checks and confirms them; only then are the phase's checkboxes marked done and the commit made. A green automatic gate is explicitly not, by itself, authority to commit a phase with outstanding manual checks.

9.3 Version control conventions

Git tracks every change. One commit corresponds to one phase, made only once that phase's gate is green and any manual checks it carries have been confirmed by the owner. Commit messages follow a fixed convention — a prefix naming the affected area, a concise imperative summary, and a body explaining what changed and why whenever the summary alone is not enough. Paths are staged explicitly; nothing is ever staged with a blanket `add -A`. Nothing is ever pushed, and no release tag is ever cut, without an explicit instruction from the owner.

10 Manual verification

Automatic tests cannot fully exercise the terminal layer, since several behaviours — real window resizing, real terminal-emulator rendering quirks, and subjective legibility — depend on a live controlling terminal that a test harness cannot faithfully simulate. The project keeps a dedicated manual testing guide for this reason, organized by the same module boundaries as the architecture itself, so a change to one module points directly at the section of manual checks it should also motivate:

Area changed	Manual checks to run
<code>out.c</code> , <code>width.c</code>	Output modes, screen width, CJK/combining-character alignment
<code>menu.c</code> , <code>comp.c</code> , <code>sqlctx.c</code>	Completion menu behaviour
<code>hl.c</code> , <code>theme.c</code>	Syntax highlighting, theme loading and legibility
<code>edit.c</code> , <code>line.c</code>	Line editor, keybindings, pty-driven behaviour
<code>dot.c</code> , <code>schema.c</code> , <code>import.c</code> , <code>shell.c</code> , <code>main.c</code>	Dot-command and CLI parity, plus make parity

Representative checks from that guide illustrate the kind of behaviour being protected: that a completion menu narrows in place rather than closing when the user keeps typing; that `Esc` restores the input buffer to exactly its pre-`Tab` state; that an unknown column name in a `SELECT` list is left in the plain foreground rather than falsely highlighted as known; that `-screenwidth 1` is rejected with a specific exit code rather than silently clamped; and that killing the shell mid-line with `SIGTERM` leaves the terminal in normal, non-raw mode without requiring the user to run `reset`.

11 Using sqlsh

11.1 Building

The entire toolchain is pinned by a Nix flake; no system dependency beyond `libsqlite3` is assumed.

```
nix develop          # enter the dev shell once, then run make from inside it
make                # debug build -> bin/sqlsh
make release        # optimised build
make SQLITE=vendored # build against the SQLite amalgamation instead
make test           # unit + pty tests under ASan+UBSan
make parity         # differential output test vs the real sqlite3(1)
make gate           # format, both builds, tests, parity, cppcheck, clang-tidy
make install PREFIX=~/.local
```

`make help` lists every target, including `valgrind`, `fuzz`, `tidy`, `cppcheck`, `compdb`, and `watch`. Without Nix, `nix build` — using the flake’s default package — produces the same release binary.

`make install` places `bin/sqlsh` and its man page only; it never installs a symlink or copy named `sqlite3`, leaving that substitution to the user’s own `alias`.

11.2 Basic usage

```
sqlsh mydb.db          # interactive, completion + highlighting + box on
sqlsh --compat mydb.db "SELECT * FROM t;" # scripting, byte-identical to sqlite3(1)
alias sqlite3=sqlsh     # meant to survive a normal day's work
```

With no database argument, `sqlsh` opens an in-memory database, exactly as `sqlite3(1)` does. Any SQL given on the command line after the database name runs as a batch, after which the shell exits; with none given, it opens the interactive prompt, or reads statements from standard input when that is not a terminal.

11.3 Keybindings

Two keymaps are available, switched at runtime with `.editor emacs` or `.editor vi` — no restart required. Emacs is the default.

Emacs keymap (always available)

Key	Action
← → / Ctrl-B Ctrl-F	Move the cursor one character

Key	Action
Ctrl-A / Ctrl-E	Jump to start / end of line
Ctrl-W	Delete the word before the cursor
Ctrl-U	Delete to the start of the line
Ctrl-K	Delete to the end of the line
Ctrl-T	Transpose the two characters before the cursor
↑ ↓	Walk history
Ctrl-L	Clear the screen and repaint
Ctrl-C	Abandon the current line
Ctrl-D	End of file on an empty line, delete-forward otherwise
Tab	Open (or narrow) the completion menu; accept a unique match with no menu

Vi keymap (`.editor vi`), normal mode

Key	Action
h j k l	Move / history
0 \$ ^	Start, end, first non-blank of the line
w b e	Word motions
i a I A	Enter insert mode (before/after cursor, start/end of line)
x	Delete character under the cursor
d + motion (dw, dd, ...)	Delete
c + motion (cw, cc, ...)	Change
r	Replace one character
u	Undo
a leading count	Repeats the motion or command

Completion menu, once open (either keymap)

Key	Action
Tab / Shift-Tab	Next / previous candidate
← → ↑ ↓, Ctrl-N Ctrl-P	Move the selection
Typing	Narrows the candidates in place
Enter	Accept the selection
Esc / Ctrl-G	Dismiss, restoring the buffer exactly

11.4 Reference documentation

`man sqlsh (docs/sqlsh.1)` is the exhaustive reference: every flag, every output mode, every file location, every environment variable, and the exit status convention. It documents, among other things:

- `-safe` together with `-nonce STRING`, for running an untrusted script while refusing commands capable of damage outside the database file, with a nonce-prefixed escape hatch for individual trusted lines;
- `-screenwidth N` (minimum 2) to fix the width used for columnar output, overriding live terminal-width detection;
- the three-tier exit status convention: 0 on a clean exit, 1 on a SQL error under `-bail` or in the default non-interactive stop-on-error behaviour, 2 on a command-line usage error.

12 Summary

`sqlsh` demonstrates that a drop-in-compatible tool and a meaningfully better interactive tool are not in tension, provided compatibility is architecturally isolated rather than bolted on. The `--compat` flag and its differential test suite give upstream behaviour a hard boundary; everything on the other side of that boundary — the completion pipeline, the highlighter, the theme system — is free to diverge in whatever direction makes daily interactive use better, because nothing about it can leak into scripted behaviour by accident. The module layering that keeps `<sqlite3.h>` out of everything above `db.c` is what makes both halves of that claim independently testable, and the specification-driven workflow around the codebase is what keeps the architecture and the roadmap from drifting apart as the project grows.